

DIAS: Distance-based Attention Sparsity for Ultra-Long-Sequence Transformer with Tree-like Processing-in-Memory Architecture

Zekai Chen*, Yiming Chen*, Teng Wan, Tianyi Yu, Yu Wang, Huazhong Yang and Xueqing Li
Department of Electronic Engineering, LEFT/BNRist/SKLSNC, Tsinghua University, Beijing, China
 xueqingli@tsinghua.edu.cn

Abstract—Long-context inference has become a central focus in recent self-regressive Transformer research. However, challenges still remain in performing decode stage due to the memory bandwidth bottleneck of attention mechanisms and the substantial memory overhead associated with KV cache. Although attention sparsity has been proposed as a potential solution, conventional sparsity methods that rely on heuristic algorithms often suffer from accuracy degradation when applied to ultra-long sequences.

To break through the dilemma between accuracy-performance and bandwidth-capacity, this work proposes DIAS, a distance-based irregular attention sparsity approach with processing-in-memory (PIM) architecture. DIAS employs approximate top-K attention (AKAttention) scores through graph-based search to improve inference efficiency while maintaining accuracy. Furthermore, a scalable tree-like PIM (TreePIM) architecture is introduced to achieve both memory capacity and bandwidth improvement by isolating enormous memory access for KV cache into the PIM units. Evaluations on various configurations of DIAS for Longbench with Llama3-405B models with 1M sequence length show up to 75 times speedup compared with the state-of-the-art LLM accelerator, with accuracy drop of less than 1%.

Index Terms—AI and Machine Learning, Architecture & System Design

I. INTRODUCTION

Large Language Models (LLMs) built on the attention-based Transformer architecture [1] have demonstrated remarkable potential and achieved significant advancements across computer vision (CV) [2], natural language processing (NLP) [3], and automatic control [4]. Especially, emerging applications, such as long-term agency, retrieval-augmented generation (RAG) [6], and multimodal processing [7], underscores the growing demand for long-context LLM inference, with sequence length requirements increasing from approximately 1K to beyond 10M.

However, it is challenging to deploy Transformer-based LLM in long-context scenarios in terms of memory bandwidth and capacity. As shown in Figure 1 (a), GB-level KV cache has to be totally loaded to generate the next token, which results in serious memory bandwidth and memory capacity bottleneck. What's worse, the memory bandwidth and capacity is a dilemma in physic. Existing scale-out scheme for expansion is severely constrained due to inter-chip communication, which incurs significant performance degradation.

* : Both authors contributed equally to this research.

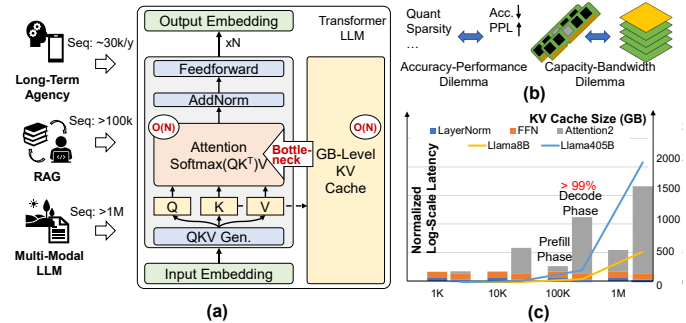


Fig. 1: The long-context LLM inference challenges illustration of (a) memory bandwidth bottleneck due to KV cache, (b) accuracy-performance and capacity-bandwidth dilemma, and (c) the latency breakdown and memory capacity trends.

Fortunately, various solutions for the long-context LLM inference have been proposed, including quantization [8] [9], speculative decoding [10] [11], alternative attention mechanism [12] [13], and attention sparsity [14] [15]. Quantization, while compatible with other techniques, inevitably leads to accuracy degradation, as shown in Figure 1 (b), and can result in significant fine-tuned knowledge loss in industrial LLMs. Furthermore, speculative decoding can accelerate inference by utilizing the multiple consecutive inference results of a small model. But this approach highly depends on the capacity of small models and could increase the decoding energy cost and delay. Alternative model architectures, such as Mamba [33] and TTT [34], have been proposed as potential replacements for Transformers. However, these emerging structures introduce significant information loss in long-text scenarios due to single hidden state compression. Thus far, sparsity exploitation remains the most promising and effective approach for addressing the memory-bond in long-context inference.

Numerous works have attempted to leverage the sparsity in attention computation shown in Figure 2. Recent work ToPick [14] could early eliminate the K cache retrieving by calculating the probability of attention score. However, the compression rates is still limited due to the pruning method. What's more, heuristic sparse methods like Window Attention [15], Stream LLM [16], H₂O [17] are proposed to discard part of KV cache based on different importance prediction strategy. For instance,

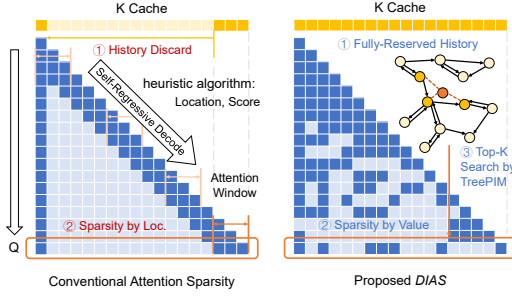


Fig. 2: Illustration of the bottlenecks of conventional attention sparsity [15] [16] and the innovations of proposed DIAS.

Window Attention only utilizes local context windows, which completely loss the history information. Other methods, such as inflLM [18], attempt to offload KV cache from GPU memory to CPU memory by selecting representative keys for different groups and retrieve KV cache with look up table. Although these methods claim efficiency, there remains significant accuracy drop in ultra-long-context scenarios.

To tackle the aforementioned problems, we proposed DIAS, a distance-based irregular attention sparsity for long-sequence P/D separated transformer acceleration. As a software-hardware co-design solution, DIAS focuses on the bottleneck in decode phase of self-regressive LLM to break through the accuracy-performance and bandwidth-capacity dilemma. The innovations includes: (1) a spatially structured and dynamic updated representation of KV cache, (2) an effective attention sparsity computing method based on approximate top-K value to achieve performance improvement with accuracy maintaining, and (3) TreePIM architecture based on switches to achieve high bandwidth with large capacity. Notably, this work is compatible with recent researches on attention modifications, such as decoder-decoder model [20], razor attention [21], making DIAS applicable widely. Additionally, this method could be further researched and utilized in ultra-long-context scenarios to tackle attention noise problems and the bottleneck of model pretraining length, both of which have emerged as central challenges in the field of LLM algorithms

The major contributions of this paper are as follows:

- A distance-based irregular attention sparsity algorithm AKAttention in attention computing with almost no accuracy drop to address the memory bandwidth bottleneck due to KV cache retrieve.
- An effective hardware architecture TreePIM is presented for balancing the gap between memory bandwidth and capacity by processing-in-memory to address the dilemma between memory capacity and bandwidth.
- Rich experiment results shows that the proposed DIAS achieves about x25.3-171 speedup and x1.7-5.0 energy efficiency compared with conventional works.

II. BACKGROUND

A. Attention Mechanism

The key contribution of Transformer [1] is the self-attention mechanism, which enables the model to capture long-range dependencies between elements in a sequence, thereby enhancing its ability to model complex patterns.

As is shown in Figure 1, in a typical Transformer layer, each input token undergoes a series of transformations through multiple sub-layers, the most important of which is the multi-head self-attention mechanism. For a given input sequence of tokens, represented as embedding $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]$, self-attention model computes as follows:

$$O = softmax(\frac{XW_Q \cdot (K_{Cache}, XW_K)}{\sqrt{d_k}}) \cdot (V_{Cache}, XW_V)$$

It could be found that attention computing is a typically memory-intensive application. The memory access and computation complexity grows linearly with sequence length, which results in serious performance bottlenecks.

B. ANNS Algorithms

Approximate nearest neighbor search (ANNS) algorithms are essential for accelerating large-scale search tasks by efficiently identifying approximate neighbors in high-dimensional spaces. These methods are particularly valuable in applications like similarity search and are widely used in large-scale machine learning systems, such as recommendation system[li2019approximate].

ANNS algorithms have been extensively researched, with each approach offering distinct advantages suited to different scenarios. Well-established methods such as KD-Tree [23], Product Quantization (PQ) [24], and Inverted File (IVF) [25] are widely used for efficient nearest neighbor search. Graph-based methods, however, are considered state-of-the-art for ANNS due to their high efficiency and accuracy. Hierarchical Navigable Small World (HNSW) graphs [26], for example, is one of the most effective and widely used graph-based ANNS methods. More recently, CAGRA [27] has emerged as an advancement over HNSW. CAGRA is designed to leverage the parallel processing capabilities of GPUs, achieving substantial improvements in speed and scalability compared to traditional CPU-based approaches.

C. Processing-in-Memory (PIM) Architectures

Processing-in-Memory (PIM) architectures have become a key solution for reducing data movement in data-intensive tasks. In Transformer models, PIM approaches like ReTransformer [28] use ReRAM-based memory to speed up attention calculations, while TransPIM [30] combines PIM with High Bandwidth Memory (HBM) for faster processing. More recently, AttAcc [36] and NeuPIMS [37] have delved into architecture design and in-memory parallelism. Furthermore, SpecPIM [38] leverages a PIM architecture to speed up speculative decoding in language models.

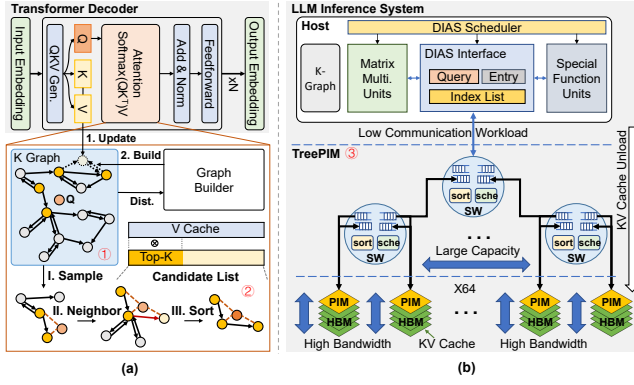


Fig. 3: Proposed DIAS: (a) irregular attention sparsity based on top-K score search algorithm, and (b) TreePIM architecture for graph-based sparsity attention acceleration.

For Approximate Nearest Neighbor Search (ANNS), PIM architectures like NDSearch [29] and Pyramid [31] leverage near-data processing to optimize graph traversal, introducing different designs in dataflow, hierarchy and memory management methods. These PIM designs are critical for scaling Transformer models and ANNS tasks in large applications, providing high throughput and reduced energy costs.

III. PROPOSED WORK

A. Overall Architecture DIAS

The overall architecture is depicted as Figure 3. The P/D separation technique is adopted in this work to focus on the decode phase of self-regressive LLM. This work proposed (1) graph-structure dynamic-updated K Cache, (2) irregular attention sparsity based on top-K attention score search, and (3) tree-like PIM architecture for isolating the high data communication into the separated PIM units to achieve both large memory capacity and high bandwidth.

Firstly, a K-graph is built based on K Cache during the prefill stage on the host device. While the large KV cache is unloaded to the PIM device. At the costly decode stage, new query, key, and value vectors are computed on host by matrix multiply unit (MMU). Then, DIAS scheduler would iteratively generate and dispatch query batches based on the K-graph. Switch units (SW) are organized as a binary tree to support scalable capability and will route query batches to corresponding PIM units. The PIM units effectively compute the inner production benefited with high parallelism and DRAM bandwidth, while inner product results will be gathered to Host by SWs and sorted with parallel bitonic sort algorithm. After several iterations, Top-K attention score of $\text{softmax}(QK^T)$ will be calculated. and finally be scattered to the PIM unit to complete the sparse attention computation with V Cache.

Moreover, graph update is also performed at the same time. Graph update is performed on host and KV vectors are uniformly assigned to PIM units to keep TreePIM balance and highly utilized. Both operations are time-overlapped and hidden from the attention computation part.

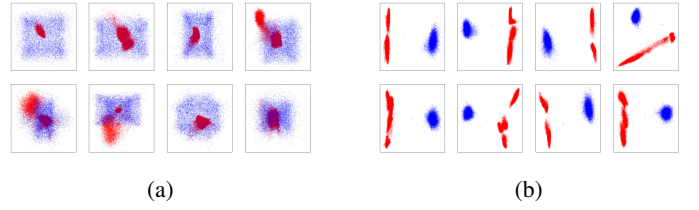


Fig. 4: Illustration of distribution on (a) separated dims, and (b) PCA with red refer to query and blue refer to key.

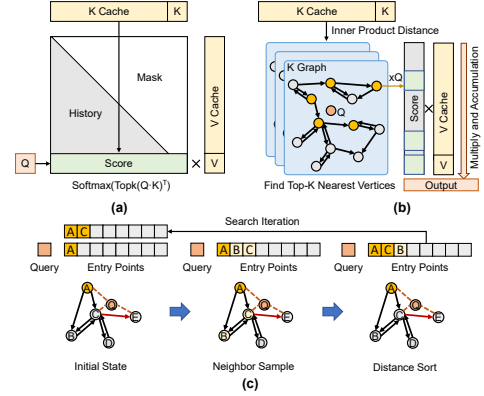


Fig. 5: Distance-based sparsity attention computing (a) principle, (b) illustration of the method, and (c) workflow.

The proposed DIAS achieves an $O(1)$ complexity and constant memory access when generating a token in Transformer-based LLMs. In contrast, conventional solutions generate one token in the conventional Attention layer with a complexity of $O(L)$ and increasing memory access, making the total complexity of generation $O(L^2)$, where L represents the sequence length.

B. AKAttention Algorithm

1) *Theoretical Analysis*: Figure 8 visualizes Llama3.1 as a sample of query, and key distributions. It shows disperse key vectors and centralized query vectors, which motivates the sparsity attention by ignoring the unrelated key cache. In contrast to conventional given keys pruning methods, this work proposes a graph-based technique to search the top-K related key cache for each query. Considering the attention score $S = \text{softmax}(QK^T)$, the relativity between query and key cache could be represented as the inner product distance.

As shown in Figure 5(b), with the transitivity of inner product, a graph built by key cache (K-Graph) could be constructed with inner product distance to represent the relativity between tokens and perform efficient attention computation with graph based ANNS algorithm integrated in work flow.

2) *Initialization*: The K-graphs of AKAttention is initialized in the prefill stage. Redundant edges in K-graph is pruned motivated by CAGRA [27] using detourable route counts, making each vertices with a fixed out-degree to fully utilize the parallelism. The entry points are set to recent visited or

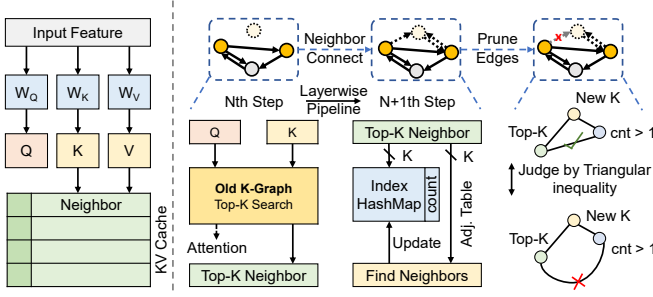


Fig. 6: Dynamic K-Graph update method in the proposed AKAttention.

updated vertices cached on host, while generated KV cache are uniformly offload to TreePIM as interleaved or random mode.

3) *AKAttention Workflow*: Figure 5(c) shows how AKAttention finds the top-K attention scores after the new key, value, and query generation:

At the first step of iteration, AKAttention computes the inner product between the entry points of K cache and the new query. The results will be sorted and the top-K largest keys will be reserved. At the second step of iteration, the neighbors of the reserved keys in the K-graph is sampled from key graph as the new entry points.

After reaching the maximum number of iterations, AKAttention will compute the attention scores by softmax the inner production results and multiply with the corresponding value cache to generate the final attention computing results.

Notably, the smallest value in top-K queue is cached in the host to prune useless candidates and their inner product result.

4) *K-Graph Dynamic Update*: The dynamic update method for K-graph is illustrated as Figure 6.

During the decode phase, the new key in K-graph is firstly connected to top-K neighbors, which is searched together with the new query in the AKAttention. Then detourable routes are counted using the same criteria in proxy graph initialization and edges with the most detourable routes are pruned. With index hash map, this process is performed $O(1)$ complexity, which could be fully parallel among different layers.

The detourable criteria used in AKAttention is: $(e_{X \rightarrow Z}, e_{Z \rightarrow Y})$ is a detourable route if $\max(w_{X \rightarrow Z}, w_{Z \rightarrow Y}) < w_{X \rightarrow Y}$

C. TreePIM Architecture

1) *TreePIM Structure*: The proposed AKAttention provides an approach to maximum utilizing the sparsity in attention computing. Furthermore, an processing-in-memory architecture TreePIM is proposed to cope with the dilemma between the memory bandwidth and memory capacity.

As depicted in Figure 7, TreePIM is a tree-like scalable PIM architecture utilizing the relatively small communication cost of sending indices and inner production results in AKAttention to achieve ultra high memory bandwidth with scalable large memory capacity in AKAttention computation. TreePIM is

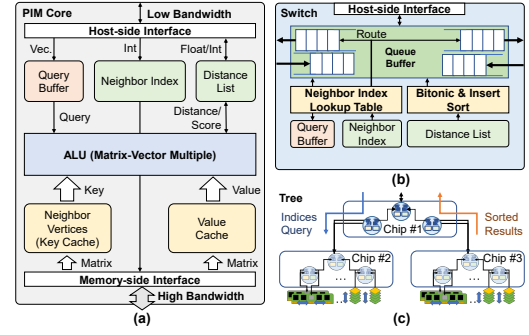


Fig. 7: Circuits design of (a) processing-in-memory unit for effectively sparsity attention computing, and (b) switch unit for (c) dynamic scalable capability.

composed by PIM devices and switches while switches connect in a tree-like mode while the PIM devices are organized as leaves.

The low-cost indices and inner production results is passed through the switch tree, where the high-bandwidth key-query inner production are performed parallel in PIM devices, boosting the total memory bandwidth in the system level.

Also, TreePIM is internally flexible with tree-like topology, meeting the large variation of memory budget in different inference scenarios ranging from 8B Model 10k tokens to 405B Model with 10M tokens.

2) *PIM Unit*: The PIM unit used in this work shows as Figure 7(a). As only inner product and weighted sum are involved in PIM computation, the proposed PIM unit is relatively simple, resulting in small area and energy overhead.

When the PIM unit receives query vectors and entry point indices, query vectors are cached in the query buffer and reused in the following top-K search iterations. PIM utilizes high bandwidth communication to read the KV cache vectors from memory and compute inner production, then send the lightweight result back through switches.

3) *Switch*: The switch unit shows as Figure 7(b), consisting of lookup tables and sort units. When a switch unit receives packages from its father node, it redirects the package to left or right according to the lookup tables. The package will be passed to both left and right if the lookup tables miss. The lookup table is dynamically updated using the return results from its left son and right son.

The sort part works when results are returned from left son and right son. In switch, high parallelism sort is performed utilizing the bitonic sort algorithm. Insert sort is performed if a switch is connected to PIM unit to make a bitonic sequence.

4) *Workload Balance*: To deal with different scenarios, '-S', '-L', '-LL' memory manage modes are proposed.

S mode is designed for relatively small KV cache. In the S mode, graph structure and location of each key are stored on host. In this scenario, each package is assigned with a certain target dram and switch could transfer based on it. And sampling is conducted on host which alleviate the communication cost between switch.

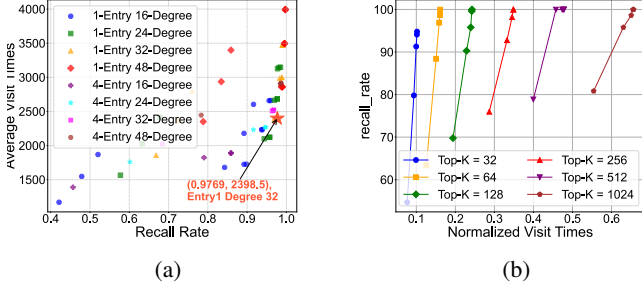


Fig. 8: (a)Top-K search average retrieved vertices and corresponding recall rate on different numbers of search entry and graph degrees (b) Analysis of recall rate on different iteration times and top-K setup.

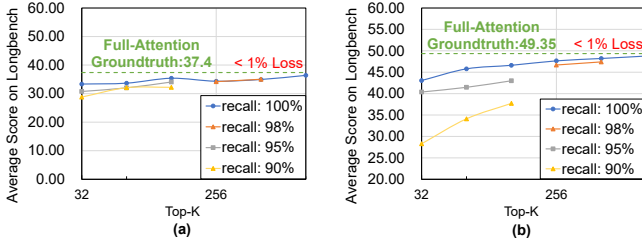


Fig. 9: Performance variation with top-K and recall rate of Longbench on (a) Mistral-7B, and (b) Llama3.1-8B.

L mode is designed for larger KV cache size. In L mode graph structure is stored in TreePIM, and retrieved each time with indices and inner product results. Only the neighbor lists of updated top-K vertices and top-K vertices for new generated key vector are stored on host, which will be used for sampling and update in next layer computation respectively.

In LL mode, location table of each key are also abandoned to alleviate memory cost. Lookup table is used to instruct transfer process in switch and candidate broadcast could lead to more transfer cost.

Notably, KV pairs are uniformly assigned to the DRAM devices in TreePIM with interleave or random method. This mechanism ensures load balancing and system efficiency.

IV. EXPERIMENT

A. Experiment Setup

1) *Software setup*: PyTorch is used as the Transformer-based LLM framework for the tasks inference of the proposed DIAS architecture on 8 NVIDIA A100. The proposed AKAttention is adopted as the attention sparsity mechanism. The LLM structure used in experiments is the widely used Llama3.1 and Mistral. All models are well pretrained large model. To evaluate the advantages of AKAttention in long-context scenario, the Longbench [32] is selected as benchmark, which includes 21 datasets in different scenarios. The datasets contains prompts with average length approximate to 10K, and maximum length up to 40k, which is fairly evaluated

TABLE I: System-Level Performance Comparison

	DIAS (This Work)	Vanilla	Window Attention	ToPick DAC'24
Config	4 * 16 32GB HBM	HBM-PIM	16GB HBM2	
Model&Seq Len	Llama3.1-405B&1M			
Acc. Drop	<1%	Full	>30%	Negligible
Throughput (Token/min)	461.5	2.705	45.801	6.21
Energy Eff. (Token/kJ)	1.7	0.42	4.1	1.68
Speedup	171.7	1	16.9	2.3

the accuracy impact of different attention sparsity methods in varied long-sequence LLM application scenarios.

2) *Hardware setup*: The performance of PIM units and switch units in TreePIM architecture are based on DC simulations in a 28nm CMOS process. The memory and peripheral latency and energy consumption are obtained from commercial memory and bus standard, DDR, HBM, and PCIE. In the system-level experiment, two configurations of DIAS are used for Llama3.1-8B decoding from 100k tokens scenario and Llama3.1-405B decoding from 1M tokens scenario. In 100k-8B model scenario, DIAS decodes with top-1K tokens utilizing 8-PIM-units 3-level TreePIM managed in S mode. For the other scenario, DIAS decodes using top-3K tokens with 4 channel of 16-PIM-units 4-level TreePIM managed in L mode.

3) *Baseline setup*: In the system evaluation, vanilla is assigned with conventional attention mechanism and typical loop-reduced-based scale-out architecture. TreePIM-Only performs conventional attention mechanism in TreePIM architecture, and AKAttention-Only performs AKAttention with conventional architecture. DIAS denotes the proposed design combined with AKAttention and TreePIM. The software evaluation compares the accuracy on the proposed AKAttention of 128-top-K and 1k-top-K with previous attention sparsity methods: Streaming LLM [16], InfLLM [18], Infinite [35], H2O [17]. The system-level performance is compared with vanilla, slide window attention [35], and SOTA LLM accelerating architecture ToPick [14] utilizing retrieve pruning.

B. Experiment results

1) *Top-K retrieving analysis*: Figure 8a shows the relationships between average number of retrieved vertices in searching and the corresponding recall rate in AKAttention of different configurations. It could be found that there is a significant increase in recall rate as out-degree of the K-graph grows, which is useful for improving the inference accuracy with attention sparsity. However, the average retrieved vertices, which represents the overhead of AKAttention also increased. On the other hand, 1 entry for 1 top-K out performs 4 entry in both recall rate and efficiency. Considering the trade-off between performance and accuracy, this work suggests the use of 1-Entry 32-Degree to achieve the best system-level throughput with negligible accuracy drop.

Further, analysis on the selection of top-K is discussed in Figure 8b. It shows that the proposed AKAttention in DIAS could achieve over 99% recall rate on most of the top-k based

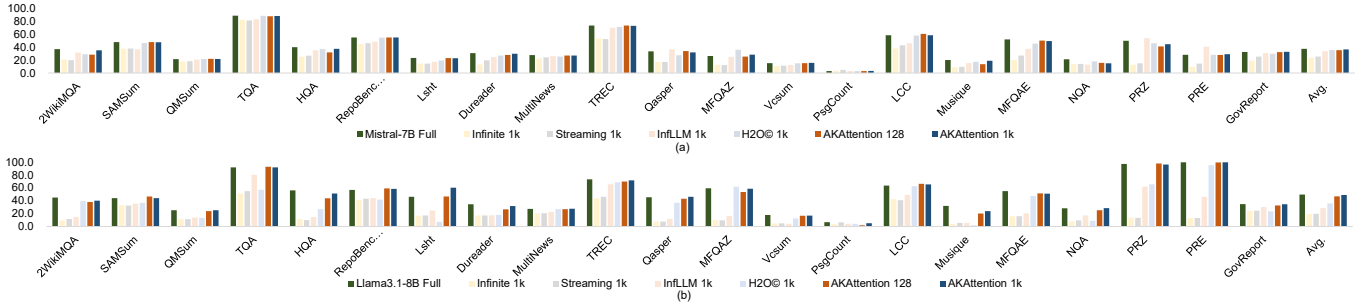


Fig. 10: Accuracy comparison between the proposed DIAS and conventional sparsity methods on Longbench with (a) Mistral-7B, and (b) Llama3.1-8B.

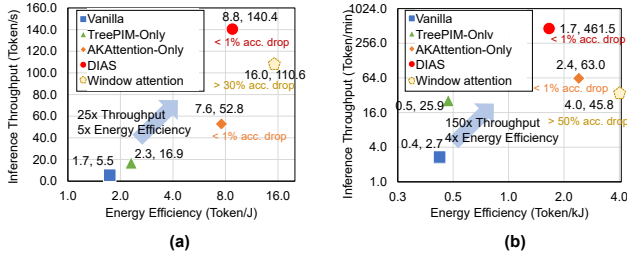


Fig. 11: System-level inference throughput and energy efficiency simulation results with (a) 100k seq len on Llama3.1-8B, and (b) 1M seq len on Llama3.1-405B.

on the previous configuration. In this work, the selection of top-K is a configurable parameter that depends mainly on the accuracy and performance needs of the tasks. As shown in Figure 9, an accuracy drop less than 1% could be achieved on both Mistral-7B and Llama3.1-7B, with a proper top-K.

2) *End-to-end accuracy evaluation*: Figure 10 shows the accuracy comparison between the proposed AKAttention in DIAS and the conventional sparsity methods on the long-context dataset Longbench. As shown in the figure, the proposed AKAttention achieves less than 1% accuracy drop or even better on most tasks in Longbench even with the top-128 setup configuration, which is much smaller than the other methods that could lose more than 30% of accuracy.

3) *System-level performance comparison*: Table. I shows the performance comparison with conventional architecture and the prior SOTA ToPick [14]. Despite extreme energy efficiency of Window Attention, its accuracy degradation in long-sequence scenarios is lethal. ToPick achieves about 2x efficiency improvement by employing prediction in K cache retrieving. However, the improvement is still limited due to the constraint of the possibility calculation.

Fig.11 visualize the performance comparison between various scheme. In 1M tokens Llama3.1-405B scenario, DIAS achieves about 170x speedup and 1.7x energy efficiency improvement. While in 100k tokens Llama3.1-8B scenario, DIAS achieves 25x speedup and 5x energy efficiency improvement. Compared with window attention algorithm of over 30% accuracy loss, the proposed DIAS effectively utilizes the sparsity in

attention with less than 1% accuracy degradation. DIAS shows great potential on long-context LLM inference and could be compatible to many other techniques.

V. OVERHEAD DISCUSSION

Although this work achieves an extremely high throughput improvement using a Top-K based irregular sparse attention mechanism, a large number of inter-switch transfers become a new bottleneck. inter-switch transfers k cache indices with a relatively small amount of data. However, the bandwidth of switch units remains a bottleneck under the scalability constraint due to the low dimension of each header in the current large model architecture, which is only 128. There are several potential mitigation schemes for this problem, including expanding hidden size of each head, index list compression and packetized fetching of multiple headers. These will be further investigated in the future work.

VI. CONCLUSION

In this paper, a scalable, high-throughput PIM architecture DIAS for long-context LLM inference is proposed to deal with the memory bandwidth bottleneck in attention computing. DIAS utilizes approximate near neighbor search (ANNS) to find the top-K relative key cache with the given query, which greatly alleviates the memory bandwidth challenges. On the one hand, AKAttention only needs to fetch a small portion of the key cache, reducing the total communication volume. On the other hand, TreePIM isolates the high-bandwidth key retrieving into the PIM region with scalability. The communication inter-chip is reduced by hundreds times compared with conventional loop-reduced scale-out architecture. The system-level evaluation shows that the proposed DIAS achieves up to 75 times speedup compared with the SOTA LLM accelerator, with less than 1% accuracy degradation.

VII. ACKNOWLEDGMENT

This work was supported in part by National Key R&D Program of China (2024YFB3214500) and by NSFC under Grant U21B2030, Grant U24B6015, Grant 92264204 and Grant 62325405.

REFERENCES

- [1] A. Vaswani, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.
- [2] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, et al., "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [4] F. Giuliari, I. Hasan, M. Cristani, and F. Galasso, "Transformer networks for trajectory forecasting," in *2020 25th International Conference on Pattern Recognition (ICPR)*, 2021, pp. 10335–10342.
- [5] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2022, pp. 27268–27286.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, D. Kiela, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [7] V. Gabeur, C. Sun, K. Alahari, and C. Schmid, "Multi-modal transformer for video retrieval," in *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IV*, vol. 12349, 2020, pp. 214–229.
- [8] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length LLM inference with KV cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.
- [9] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for KV cache," *arXiv preprint arXiv:2402.02750*, 2024.
- [10] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the International Conference on Machine Learning*, 2023, pp. 19274–19286.
- [11] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, "Online speculative decoding," *arXiv preprint arXiv:2310.07177*, 2023.
- [12] P. D. Letourneau, M. K. Singh, H. P. Cheng, S. Han, Y. Shi, D. Jones, et al., "PADRe: A unifying polynomial attention drop-in replacement for efficient vision transformer," *arXiv preprint arXiv:2407.11306*, 2024.
- [13] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," *arXiv preprint arXiv:2001.04451*, 2020.
- [14] J. Park, M. Kang, Y. Han, Y. G. Kim, J. Shin, and L. S. Kim, "Token-pick: Accelerating attention in text generation with minimized memory transfer via probability estimation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [15] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10012–10022.
- [16] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023.
- [17] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, et al., "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 34661–34710.
- [18] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, et al., "InFLM: Training-free long-context extrapolation for LLMs with an efficient context memory," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [19] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [20] Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, and F. Wei, "You only cache once: Decoder-decoder architectures for language models," *arXiv preprint arXiv:2405.05254*, 2024.
- [21] H. Tang, Y. Lin, J. Lin, Q. Han, S. Hong, Y. Yao, and G. Wang, "RazorAttention: Efficient KV cache compression through retrieval heads," *arXiv preprint arXiv:2407.15891*, 2024.
- [22] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1475–1488, 2019.
- [23] J. L. Bentley, "K-d trees for semidynamic point sets," in *Proceedings of the Sixth Annual Symposium on Computational Geometry*, 1990, pp. 187–197.
- [24] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2010.
- [25] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Computing Surveys (CSUR)*, vol. 38, no. 2, pp. 6-es, 2006.
- [26] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [27] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, "CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs," in *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 4236–4247.
- [28] X. Yang, B. Yan, H. Li, and Y. Chen, "ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, 2020, pp. 1–9.
- [29] Y. Wang, S. Li, Q. Zheng, L. Song, Z. Li, A. Chang, and Y. Chen, "NDSEARCH: Accelerating graph-traversal-based approximate nearest neighbor search through near data processing," in *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 368–381.
- [30] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 1071–1085.
- [31] Z. Zhu, J. Liu, G. Dai, S. Zeng, B. Li, H. Yang, and Y. Wang, "Processing-in-hierarchical-memory architecture for billion-scale approximate nearest neighbor search," in *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [32] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," *arXiv preprint arXiv:2308.14508*, 2023.
- [33] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.
- [34] Y. Sun, X. Li, K. Dalal, J. Xu, A. Vikram, G. Zhang, and C. Guestrin, "Learning to (learn at test time): RNNs with expressive hidden states," *arXiv preprint arXiv:2407.04620*, 2024.
- [35] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang, "LM-Infinite: Simple on-the-fly length generalization for large language models," *CoRR*, vol. abs/2308.16137, 2023.
- [36] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! Unleashing the power of PIM for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, vol. 2, 2024, pp. 103–119.
- [37] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, vol. 3, 2024, pp. 722–737.
- [38] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, "SpecPIM: Accelerating speculative inference on PIM-enabled system via architecture-dataflow co-exploration," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, vol. 3, 2024, pp. 950–965.